

A Numerical $2 + 2$ Form of Chan's Four-Dimensional Hypervolume Algorithm

Finite weighted chains, pairwise summation, and an explicit recursion

Research working report

4 October 2026

Abstract

We give an exact numerical formulation of four-dimensional anchored hypervolume and dominated-set ℓ_1 magnitude with $\tilde{O}(n^{4/3})$ arithmetic complexity. The exponent is Chan's; the purpose is a simpler implementation and a transparent recurrence. Each coordinate is replaced by a finite ordered list of cells carrying numerical masses. The state consists only of numerical unary arrays, integer monotone cutoff arrays, and scalar coefficients. Eliminating a variable uses a prefix sum and deterministic choices of its active lower and upper bounds. Compression stores block *masses*, so it needs neither symbolic integration nor average densities or division by cell measure. Opposite monotonicity directions remain separate, avoiding finite-run inversion and special endpoint decorations.

Two coordinate eliminations form the numerical pair operation. Two pairs of binary cuts form a geometric round: at most sixteen descendants have one eighth of the parent's weighted face complexity. Locally chosen blocks give $O(\log \log n)$ compression generations; a constant raw emission factor per generation is consequently only polylogarithmic. This explicitly accounts for representation growth. A standalone Python prototype and exact verification accompany the report. The result is an arithmetic-complexity formulation, not a claim of competitive floating-point performance or an improvement over Chan's exponent.

Contents

1	The target and the simplification	3
2	From product measure to finite weighted chains	3
3	The numerical state and its invariant	4
4	The only numerical elimination kernel	5
4.1	Comparing two unary bounds	5
4.2	Summing one variable	5
4.3	The $2 + 2$ operation	6
5	Compression by block mass, without averaging	6
6	The geometric recursion, step by step	8
6.1	Absorb the easy boxes	8
6.2	Two pairs of binary cuts	8
6.3	Purely integer median weights	9
6.4	The block schedule	9
7	Pseudocode that separates the two recursions	10
8	Correctness and the complete work bound	11
8.1	Correctness	11
8.2	One block, with term growth included	11
8.3	Solving the recurrence explicitly	11
9	A worked four-point example	12
10	Magnitude and hypervolume in both directions	13
11	Relation to the supplied repositories	13
12	Implementation and verification	14
13	Numba implementation and measured comparison	15
14	What extends to dimensions six and eight	16
15	The resulting algorithmic target	16

1 The target and the simplification

For a finite set $P \subset \mathbb{R}_{\geq 0}^4$, write

$$D(P) = \bigcup_{p \in P} \prod_{i=1}^4 [0, p_i].$$

The two requested quantities are

$$\text{HV}_4(P) = \lambda_4(D(P)), \quad \text{Mag}(D(P)) = \left(\bigotimes_{i=1}^4 (\delta_0 + \tfrac{1}{2}\lambda) \right) (D(P)).$$

The second equality is the product-measure form of the dominated-set magnitude formula in Emmerich [2]. Empty input has value zero for both quantities. Zero coordinates are permitted and matter for magnitude.

Chan's orthant algorithm gives the $\tilde{O}(n^{d/3})$ bound for fixed $d \geq 4$ [1]. Its essential ingredients are absorption of locally one- and two-sided orthants, weighted geometric cuts, and periodic compression. We retain these ingredients. The continuous function algebra is replaced by finite tables and exact summation.

Theorem 1 (Numerical four-dimensional formulation). *There is an algorithm for $\text{HV}_4(P)$ and $\text{Mag}(D(P))$ using $O(n^{4/3} \log^c(n+2))$ arithmetic operations, for a fixed constant c . Its operations are sorting, comparisons, integer-index searches, and arithmetic on numerical arrays. It uses no symbolic integration or numerical quadrature. It does not construct a four-dimensional product grid.*

The proof includes representation growth and compression work, not just the number of geometric leaves. Arithmetic operations have unit cost; the theorem is not a bit-complexity or numerical-stability bound. Here $\tilde{O}$, or O^* in the sense intended in the question, suppresses polylogarithmic factors and fixed-dimensional constants.

The principal simplifications are these:

1. Store one mass for each one-dimensional cell, including the anchor.
2. Keep four monotonicity/comparison classes per coordinate pair.
3. Replace every one-dimensional antiderivative by an array prefix sum.
4. Compress by summing mass into blocks; do not reconstruct a density.
5. Allow polylogarithmically many terms and prove their total cost.

The active-bound choices and compression itself remain necessary parts of this construction. Merely deleting them would not preserve the proof.

2 From product measure to finite weighted chains

On axis i , sort the distinct positive input coordinates:

$$0 = c_{i,0} < c_{i,1} < \cdots < c_{i,m_i}.$$

Use the cells

$$C_{i,0} = \{0\}, \quad C_{i,k} = (c_{i,k-1}, c_{i,k}] \quad (1 \leq k \leq m_i).$$

For a measure $\alpha_i\delta_0 + \beta_i\lambda$, give these cells masses

$$a_i(0) = \alpha_i, \quad a_i(k) = \beta_i(c_{i,k} - c_{i,k-1}). \quad (1)$$

Hypervolume uses $(\alpha_i, \beta_i) = (0, 1)$; magnitude uses $(1, \frac{1}{2})$. Keeping a zero-weight anchor cell for hypervolume is harmless and lets both computations use identical code.

Let $r_i(p)$ be the coordinate rank of p_i , with rank zero at the anchor. On a cell product, membership in $[0, p]$ is exactly

$$k_i \leq r_i(p) \quad \text{for every } i.$$

Thus the original continuous measure is the finite weighted sum of these membership indicators. Only the four *axis lists* are constructed; their Cartesian product is a specification, not an allocated array.

Lemma 2 (Exact discretization). *Replacing each input corner by its four ranks and using (1) preserves both requested values exactly. The total axis-list length is at most $4(n+1)$, and construction costs $O(n \log n)$.*

Proof. Every box-membership predicate is constant on each cell product. Product measure assigns that cell the product of its four one-dimensional masses. Summing over covered cells is therefore exact. Sorting and rank lookup construct the one-dimensional lists in the stated time. $\square$

All later computation uses indices. A feasible index interval $[L, U]$ is empty precisely when $L > U$. In particular, the anchor interval $[0, 0]$ is different from the empty interval $[0, -1]$. There are no open/closed endpoint flags or infinite numerical values in the computational state.

For example, coordinates 0, 2, 5 give magnitude masses $(1, 1, 3/2)$. The prefix array is $(0, 1, 2, 7/2)$. The anchor has mass 1, while indices 1, 2 together have mass $5/2$.

3 The numerical state and its invariant

Variables have finite ordered domains $E_i = \{0, \dots, s_i - 1\}$. A term is

$$T(k) = \gamma \prod_i a_i(k_i) \prod_e \mathbf{1}[k_{j_e} \bowtie_e f_e(k_{i_e})], \quad \bowtie_e \in \{\leq, \geq\}. \quad (2)$$

The coefficient γ and arrays a_i may be signed. Each cutoff f_e is an integer-valued monotone array. Endpoints -1 and s_j encode empty rays when appropriate.

Order each pair so that $i < j$. Store at most four cutoff arrays per pair:

	f nondecreasing	f nonincreasing
$k_j \leq f(k_i)$	one slot	one slot
$k_j \geq f(k_i)$	one slot	one slot

Within the same slot, upper bounds merge by pointwise minimum and lower bounds by pointwise maximum. These operations preserve the recorded monotonicity. A constant cutoff can be absorbed into the target unary array. Opposite monotonicity directions are *not* merged.

This fixed four-slot rule is important. It keeps the class unchanged after every compression; it does not merely bound a list whose permitted length might grow at the next generation. No valley or zigzag inverse is needed.

For p variables, a term has at most $4\binom{p}{2}$ pair conditions. With $M = \sum_i s_i$, its storage is $O_p(M)$. All arrays can be dense one-dimensional arrays; sparse step records are optional optimizations. A state F is a signed sum of S such terms.

The geometric recursion also carries a cell $\Gamma = \prod_i [\ell_i, u_i]$ of index ranges and a list B of unabsorbed rank corners. Its invariant is the numerical quantity

$$U(F, B, \Gamma) = \sum_{k \in \Gamma} F(k) \prod_{p \in B} (1 - \mathbf{1}[k \leq p]). \quad (3)$$

Initially $F(k) = \prod_i a_i(k_i)$. The answer is

$$\prod_i \sum_{k_i} a_i(k_i) - U(F, P, \Gamma_{\text{root}}). \quad (4)$$

After compression, F represents *coarse-cell masses*, rather than the old pointwise values. Equation (3) still has exactly the same meaning on the new domains.

4 The only numerical elimination kernel

4.1 Comparing two unary bounds

Bounds have the form (i, A) , meaning $A(k_i)$, or are constants. To impose $A(k_i) \leq B(k_j)$:

- If $i = j$, multiply the unary array by the Boolean comparison array. The truth values may alternate; this remains an ordinary unary array.
- If one side is constant, again use a unary Boolean array.
- If the variables differ, invert the monotone array on the larger-index variable by binary search. This produces one monotone cutoff array on the other variable and a $\leq$ or $\geq$ comparison.

Strict comparison uses the corresponding strict binary-search boundary. It does not introduce a new data type. With at most p variables, these operations take $O_p(M \log(M + 2))$ time. Merge scans can remove the logarithm, but are unnecessary for the target bound.

4.2 Summing one variable

Consider summing a term over index $q \in \{0, \dots, s - 1\}$. Invert every incident condition when needed. Each becomes one lower or upper bound on q , depending on at most one surviving variable. Include the constant bounds 0 and $s - 1$. Write the candidate lists as

$$L_1, \dots, L_a, \quad U_1, \dots, U_b.$$

There are at most $2(p - 1) + 1 = 2p - 1$ candidates of each kind: each neighbour supplies at most two of each kind under the four-slot representation.

The feasible indices are

$$\max_{\ell} L_{\ell} \leq q \leq \min_u U_u.$$

Let $P(t) = \sum_{q=0}^{t-1} a(q)$ for $0 \leq t \leq s$. Choose the *first* maximizing lower candidate L_ℓ and the first minimizing upper candidate U_u . The label is specified by

$$\begin{array}{llll} L_j < L_\ell & (j < \ell), & L_j \leq L_\ell & (j > \ell), \\ U_u < U_j & (j < u), & U_u \leq U_j & (j > u), \\ L_\ell \leq U_u. & & & \end{array}$$

These labels are disjoint and cover exactly the feasible assignments of the surviving indices. Every label comparison is handled by the previous subsection. On a label, the numerical result is

$$P(U_u + 1) - P(L_\ell). \quad (5)$$

Each summand is a scalar or a unary array on the variable supplying the winning bound. Multiply it into that variable's weight array. If both endpoints depend on the same variable, or one is constant, store the difference as one unary array instead of emitting two terms.

Lemma 3 (Fixed-class numerical closure). *Summing one variable of a p -variable term produces at most*

$$B_p = 2(2p - 1)^2$$

terms in the same four-slot class, in $O_p(M \log(M + 2))$ arithmetic operations and $O_p(M)$ storage.

Proof. There are at most $(2p - 1)^2$ winner labels, each emitting at most two terms by (5). Comparisons between distinct-variable monotone bounds give monotone pair conditions. Same-variable comparisons give unary arrays. Same-slot merging restores the four-slot invariant. Prefix arrays can be nonmonotone because weights are signed, but they are used only as unary *weights*, never as geometric cutoff maps. Thus this causes no failure of monotone inversion. A dimension-bounded number of array scans and searches suffices for each emitted term. $\square$

4.3 The $2 + 2$ operation

Define

$$\text{PairSum}_{i,j}(F) = \text{SumOut}_j(\text{SumOut}_i(F)).$$

A full four-variable sum is

$$\text{PairSum}_{3,4}(\text{PairSum}_{1,2}(F)).$$

This is a pairwise recursion with numerical one-dimensional kernels. It never builds an $s_i s_j$ table. The split specifies the elimination order; it does not assert that the state factors between the two pairs. All six coordinate-pair relations may remain present.

5 Compression by block mass, without averaging

Suppose m hard boxes remain. On each current axis $[\ell_i, u_i]$, put a block boundary immediately after every remaining cutoff p_i with $\ell_i \leq p_i < u_i$. These boundaries partition the axis into contiguous blocks

$$I_i(t_i) = [L_i(t_i), R_i(t_i)] \quad (0 \leq t_i < q_i), \quad q_i \leq m + 1.$$

Their endpoints are nondecreasing integer arrays. Define the new mass state

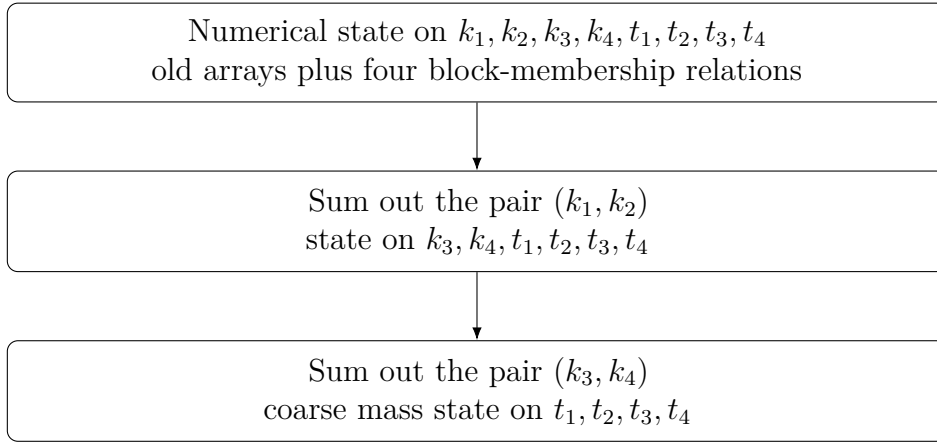
$$(\text{Push } F)(t) = \sum_{k_1 \in I_1(t_1)} \cdots \sum_{k_4 \in I_4(t_4)} F(k). \quad (6)$$

This equation specifies the answer without instructing the program to enumerate all $q_1 q_2 q_3 q_4$ block products.

Instead, attach four new variables $t_1, \dots, t_4$ with unit unary weights and append the eight pair conditions

$$L_i(t_i) \leq k_i \leq R_i(t_i), \quad 1 \leq i \leq 4.$$

The result is an eight-variable term of the same class. Sum out the old variables in the pair order (k_1, k_2) , then (k_3, k_4) . Only the four new variables remain. Rename them to the physical coordinate indices.



Lemma 4 (Numerical compression). *If the old state has S terms and total axis length M , compression emits at most $C_4 S$ terms, where*

$$C_4 = B_8 B_7 B_6 B_5$$

is a fixed constant. It costs $O(S(M + m) \log(M + m + 2))$ arithmetic operations, with a fixed dimension-dependent constant, and the output uses $O(C_4 S(m + 1))$ array entries. No product-grid enumeration or density normalization occurs.

Proof. Apply Lemma 3 four times to each lifted term. The number of variables decreases from eight to four. Its original and new axis domains have total length $M + O(m)$; elimination never introduces another domain. Each surviving array is indexed by one new variable with at most $m + 1$ entries. This proves the emission, work, and storage bounds. $\square$

Lemma 5 (Why descendants can use the compressed state). *If $g(k)$ is constant on every product of the current blocks, then*

$$\sum_k F(k) g(k) = \sum_t (\text{Push } F)(t) g(t).$$

Every remaining box mask and every subsequent cut is block-aligned. The same identity therefore remains valid throughout all descendants.

Proof. Group the finite sum by block products. Pull the constant g out of each inner sum. Every surviving box boundary was used when the blocks were built. Later cuts are selected only from boundaries of those surviving boxes, and later absorbed masks come from the same boxes. Thus no descendant requires a coordinate boundary that has been discarded. $\square$

This is the finite-sum counterpart of conditional expectation, with *mass* stored directly. A zero-mass cell causes no division by zero. The anchor may be merged with a neighbouring cell only when all remaining box masks treat them identically. Once that is true, their summed mass is all that future computation needs.

6 The geometric recursion, step by step

6.1 Absorb the easy boxes

For a box with rank corner p , first discard it if $p_i < \ell_i$ for some axis. For a remaining box, call an axis active when $p_i < u_i$. There are four cases:

1. No active axes: the box covers the cell, so return zero uncovered mass.
2. One active axis i : multiply $a_i(k_i)$ by $\mathbf{1}[k_i > p_i]$ in every term.
3. Two active axes i, j : absorb its complement as a pair condition.
4. Three or four active axes: retain the box as hard.

For a whole group of two-sided boxes on pair $i < j$, build

$$f_{ij}(k_i) = \max\{p_j : p_i \geq k_i\}, \quad \max \emptyset = -1.$$

Its uncovered condition is $k_j \geq f_{ij}(k_i) + 1$. This is a nonincreasing lower-cutoff array. Put each box endpoint in one array position, then take a suffix maximum; the group costs one scan, not one scan per box. Multiple slabs on one axis similarly reduce to their largest cutoff. Absorption does not increase the number of terms.

Unlike a continuous geometric implementation, this version need not shrink the cell for slabs. Zeroing the corresponding unary entries is sufficient. Every box retained as hard still has at least three active axes.

6.2 Two pairs of binary cuts

The cut axes cycle as

$$(1, 2) ; (3, 4) ; (1, 2) ; \dots$$

At a cut on axis i , a split after index c gives the disjoint children

$$[\ell_i, c] \quad \text{and} \quad [c + 1, u_i].$$

The answer is the sum of the two child answers. No subtraction or inclusion–exclusion across children is needed. For the face-count argument, one may embed index k as the unit cell $[k, k + 1)$. A rank cutoff p_i then has its geometric boundary at $p_i + 1$. Thus $\ell_i \leq p_i < u_i$ correctly identifies an internal boundary, even when $p_i = \ell_i$; atomic anchor cells require no exception.

For every hard box, form the triples of its active axes. Each triple represents a one-dimensional face. With positions 0, 1, 2, 3 measured cyclically from the current cut axis, assign a triple J weight

$$\omega(J) = 2^{\sum_{j \in J} \text{pos}(j)/4}. \quad (7)$$

Let Φ be the total weight. Each hard box contributes between a positive constant and another constant, so its number is $\Theta(\Phi)$. Cut at a weighted median of the cutoff coordinates of those faces whose triples contain the current axis. Ties go to the same cut; faces at that cut disappear from both children's active-face lists.

Lemma 6 (One numerical cut). *With the exact weights in (7), a child has weighted potential at most $2^{-3/4}\Phi$, after advancing the cyclic axis order. If no face uses the current axis, advance the axis without splitting; the same potential decrease holds.*

Proof. Split the parent faces into those using the current axis and those not using it. At most half the weight of the first group survives in either child. After rotating the axis order, a triple in that group changes its position sum by +1, and its individual weight is multiplied by $2^{1/4}$. The resulting total is at most $2^{1/4}/2 = 2^{-3/4}$ times its old total. For a triple not using the current axis, all three positions decrease by one, so its weight is multiplied by $2^{-3/4}$. Restricting a cell creates no new active triples. If the first group is empty, this latter argument applies to every face without a geometric split. $\square$

Corollary 7 (One 2+2 cutting round). *After cuts on axes (1, 2) followed by (3, 4), there are at most 16 descendants, each with potential at most $\Phi/8$. After q rounds there are at most r^4 descendants of potential at most Φ/r^3 , where $r = 2^q$.*

This 16-versus-8 relation is the source of the exponent $\log_8 16 = 4/3$. It is not yet a total-work proof: the numerical state also has to be charged.

6.3 Purely integer median weights

The prototype even avoids algebraic constants in selecting a median. Let n_0 be the original number of points and set $A = (n_0 + 2)^2$. For a triple with position sum $s \in \{3, 4, 5, 6\}$ use the integer weight

$$W_s = \left\lceil (A^4 2^s)^{1/4} \right\rceil.$$

Integer square roots compute its floor fourth root exactly; one comparison decides whether to round upward. Then

$$2^{s/4} \leq W_s/A \leq (1 + 1/A)2^{s/4}.$$

A median with these integer weights gives the child-potential bound $2^{-3/4}(1+1/A)\Phi$. This changes the total cut depth from $(4/3)\log_2 n_0 + O(1)$ by only $O(\log(n_0 + 2)/A) = O(1)$. Over the whole path the extra multiplicative factor is bounded by an absolute constant. Thus rounding does not change the exponent or introduce an n^ϵ loss. The combinatorial counters have $O(\log n_0)$ bits.

6.4 The block schedule

At the start of a block, let

$$Q = \left\lceil \frac{\text{sum of the current integer face weights}}{A} \right\rceil, \quad q = \max \left\{ 1, \left\lceil \frac{\log_2 Q}{16} \right\rceil \right\}. \quad (S)$$

Run q full $2 + 2$ rounds, hence $4q$ cyclic levels. At each surviving block leaf, absorb any newly easy boxes, compress onto the remaining hard grid, and start a new block using its own value of Q . The logarithm in (S) is just an integer bit-length computation.

The reason to use blocks is representation growth. Compressing after every cut could multiply the number of terms a logarithmic number of times, which would be a polynomial loss. Schedule (S) reduces the number of compression generations to $O(\log \log n)$.

7 Pseudocode that separates the two recursions

Numerical recursion.

```

PairSum(state, i, j):
    state = SumOut(state, i)
    return SumOut(state, j)

Push(state, hard-grid blocks):
    add four new block-index variables with unit weights
    add the lower and upper membership bounds for each block
    state = PairSum(state, old-axis-1, old-axis-2)
    state = PairSum(state, old-axis-3, old-axis-4)
    rename the four surviving block-index variables; return state

```

Geometric recursion.

```

Solve(state, boxes, cell, axis, levels-left):
    discard disjoint boxes; absorb all one- and two-sided boxes
    if a box covers the cell or state is zero: return 0
    if only a constant number of hard boxes remain:
        return the numerical base case
    if levels-left is zero:
        build blocks from all remaining hard-box cutoffs
        state = Push(state, blocks); relabel boxes and cell
        levels-left = 4q from schedule (S)
    find the weighted median on the current axis
    if there is no cut candidate:
        recurse on the same cell, next axis, one fewer level
    else:
        split after the median; recurse on both children
        return the sum of their answers

```

The first block is initialized by (S). Each axis advance, including a skip, consumes one level. Four advances return to the original axis order.

For a base case of at most b hard boxes, where b is a fixed constant, expand $\prod_{p \in B} (1 - \mathbf{1}[k \leq p])$ over its 2^b subsets. Each subset restricts the upper endpoint of a rectangle. Apply the two pair sums to that restricted rectangle and take the alternating sum. This is constant-size inclusion–exclusion; it is not exponential in n . The prototype defaults to $b = 2$ and uses $b = 1$ in several stress tests.

8 Correctness and the complete work bound

8.1 Correctness

Correctness part of Theorem 1. Lemma 2 gives the exact initial numerical problem. Absorption multiplies the state by the complement of each removed box, so it preserves (3). A split partitions the index cell into two disjoint sets. Every numerical elimination sums one index exactly, including ties and zero-index anchor cells. Lemmas 4 and 5 justify compression and its future reuse. Finally the base case is a finite inclusion–exclusion identity evaluated by exact pair sums. These facts prove correctness inductively; (4) recovers the covered mass. $\square$

8.2 One block, with term growth included

At a block entry let N be its weighted hard-face potential and let S be its number of terms. Apart from the first block, the axis-list length is $O(N)$ because the previous compression used only surviving hard cutoffs. The first block has $O(n)$ entries and is charged at root size.

For sufficiently large N , schedule (S) gives

$$r = 2^q = \Theta(N^{1/16}), \quad r^4 = O(N^{1/4}).$$

There are $O(r^4)$ nodes and leaves within a block. Absorptions, cuts, copies, and the compression at each block leaf each cost at most $O(SN \log^{O(1)}(N + 2))$, with a fixed constant accounting for the elimination kernel. Early numerical base cases satisfy the same charge. Thus the entire block costs

$$O(SN^{5/4} \log^{O(1)}(N + 2)). \quad (8)$$

Each child block has potential at most $c_0 N^{13/16}$ and at most $C_4 S$ terms. The constant c_0 includes integer rounding of the block length and the bounded median-weight approximation factor.

Writing $T(N, S)$ for all remaining work gives the transparent recurrence

$$\boxed{T(N, S) \leq c_1 N^{1/4} T(c_0 N^{13/16}, C_4 S) + c_2 SN^{5/4} \log^{c_3}(N + 2).} \quad (9)$$

All constants are independent of N and S . For bounded N , only a bounded number of further geometric levels is possible, and the same claim holds with a fixed-dimensional constant base cost.

8.3 Solving the recurrence explicitly

Put $\beta = 13/16$. At macro-depth g , the worst-case child size is $N_g \leq c_4 N^{\beta g}$. Consequently only $O(\log \log(N + 2))$ macro-generations occur before reaching a fixed constant size. The term count along any path is bounded by

$$S_g \leq C_4^g S = S \log^{O(1)}(N + 2). \quad (10)$$

No unproved canonical-merging bound is used here.

The number of nodes at macro-depth g is at most

$$c_5^g \prod_{h < g} (N^{\beta^h})^{1/4} = c_5^g N^{\frac{4}{3}(1-\beta^g)}.$$

Multiplying this by the per-node charge $S_g N_g^{5/4}$ gives, apart from polylogarithmic factors,

$$S N^{\frac{4}{3}(1-\beta^g) + \frac{5}{4}\beta^g} = S N^{\frac{4}{3} - \frac{1}{12}\beta^g} \leq S N^{4/3}.$$

The constants raised to g and the number of macro-levels are polylogarithmic. Summing proves $T(N, S) = \tilde{O}(S N^{4/3})$. Initially $S = 1$ and $N = O(n)$, completing Theorem 1.

This proof tolerates very large dimension-dependent emission constants. It also explains why counting leaves alone, or observing that empirical term counts stayed small, would be insufficient.

9 A worked four-point example

Consider

$$\begin{aligned} p_1 &= (1, 4, 3, 2), & p_2 &= (2, 3, 4, 1), \\ p_3 &= (3, 2, 1, 4), & p_4 &= (4, 1, 2, 3). \end{aligned}$$

Each axis list is $(0, 1, 2, 3, 4)$. The hypervolume masses are $(0, 1, 1, 1, 1)$; the magnitude masses are $(1, \frac{1}{2}, \frac{1}{2}, \frac{1}{2}, \frac{1}{2})$. Each point has exactly three active axes at the root.

On the first axis the cut candidates are 1, 2, 3, with weights $2^{5/4}, 2^{4/4}, 2^{3/4}$ respectively. The weighted median is 2. The integer weights used by the prototype select the same cut. The left index cell has $k_1 \in [0, 2]$, and the right has $k_1 \in [3, 4]$.

In the left child, p_2 is now two-sided on axes (2, 4), and p_3 is two-sided on axes (2, 3). Their complements enter the numerical state as two pair predicates. Only p_1, p_4 remain hard. In the right child, p_1, p_2 are disjoint and disappear; p_3, p_4 remain. With base threshold two, both children are already base cases.

To illustrate compression at the left child, its surviving hard cutoffs would give these index blocks:

axis	blocks
1	$[0, 1], [2, 2]$
2	$[0, 1], [2, 4]$
3	$[0, 2], [3, 3], [4, 4]$
4	$[0, 2], [3, 3], [4, 4]$

The constructor computes their mass state through the eight-variable pairwise elimination; it does not visit the $2 \cdot 2 \cdot 3 \cdot 3 = 36$ block products. This early compression is explanatory, not required by the default block schedule.

The complete numerical computation gives

$$\text{HV}_4(P) = 69, \quad \text{Mag}(D(P)) = \frac{765}{16}.$$

Both are verified independently by inclusion–exclusion over the four original boxes. The geometry and cutoff comparisons are identical in the two runs; only the initial axis masses change.

10 Magnitude and hypervolume in both directions

For nonempty P , the product measure of one anchored box is $\prod_i (1+p_i/2)$. Coordinatewise minimum describes intersections of anchored boxes and commutes with the affine map $p \mapsto \mathbf{1} + p/2$. Finite inclusion–exclusion therefore proves

$$\text{Mag}(D(P)) = \text{HV}_4(\{\mathbf{1} + p/2 : p \in P\}). \quad (11)$$

The equality also holds for empty input when its union measure is zero.

Conversely, if every coordinate of every HV corner q is at least one,

$$\text{HV}_4(Q) = \text{Mag}(D(\{2(q - \mathbf{1}) : q \in Q\})).$$

For arbitrary strictly positive HV corners, choose a common scale s with $sq_i \geq 1$ and use

$$\text{HV}_4(Q) = s^{-4} \text{Mag}(D(\{2(sq - \mathbf{1}) : q \in Q\})).$$

Boxes with a zero coordinate can first be removed for this inverse HV reduction because their four-dimensional volume is zero. They must not be removed in a magnitude computation.

The numerical engine can thus compute magnitude either directly through its atomic product masses or by the affine HV transform. The direct route is useful for testing endpoint correctness and for keeping the algebra of product measures visible. Neither direction improves the exponent by itself.

11 Relation to the supplied repositories

The following sources were inspected at pinned revisions on 4 October 2026:

- **FastHVChan**, commit 08539aa99e94, including `chan_orthant_dby3.py` and the accompanying paper [3].
- **HV4DMagnitude**, commit 0c668e765202, including `solver.py`, the audit, the closed four-dimensional report, and the pair-elimination report [4, 5].
- Its **HV468DMagnitude** code and complexity/audit reports, especially the distinction between static elimination and the $2d$ -variable compression problem [6].

The present construction is a discrete specialization of the same active-bound elimination idea. It does not claim a new hypervolume exponent or a new general elimination principle. Its useful changes are the numerical representation and the explicit closure/scheduling proof.

Continuous implementation ingredient	Finite numerical replacement
Step-function evaluation and inversion	Integer cutoff arrays and binary search
Antiderivative or threshold integral	Prefix sums of numerical unary weights
Decorated endpoints and anchor cases	One anchor cell; integer inclusive bounds
Compact finite-turn inverse	Keep the four monotone slots separate
Average density on a hard grid	Mass pushed to blocks, with no division
Symbolic breakpoint proliferation	Fixed one-dimensional index domains
Assumed small held state	Raw emission constant and $O(\log \log n)$ generations

The supplied `solver.py` is explicitly a correctness-oriented prefix-normal-form solver. It sends its live orphans into the children and rebuilds the easy state; it does not perform periodic compression. Our prototype discards absorbed boxes and carries their numerical mass state forward. Its worst-case proof charges compression explicitly. We do not infer a time bound from compression failing to trigger in particular experiments.

There is still a short structural term constructor: it enumerates winning bounds and installs array comparisons. Calling this implementation “numerical” means that all stored data and all evaluations are finite arrays and numbers. It does not mean that the necessary case distinctions have disappeared or that there is a single fixed six-number summary.

12 Implementation and verification

The accompanying standard-library Python implementation is `numerical_chan4.py`; its independent verifier is `verify_numerical_chan4.py`. The implementation is under 400 lines, including documentation and counters. Its main entry points are:

```
solver = NumericalChan4()
hv = solver.compute(points)
mag = solver.compute(points, magnitude=True)
```

Integer or `Fraction` coordinates give exact rational arithmetic. Float input gives the floating-point evaluation of the same algorithm. Signed terms can cancel; the exact-arithmetic theorem is not a promise of a small floating-point error. The prototype uses exact-key merging and normalizes nonzero unary factors for better reuse. These optimizations are not needed for the raw-emission proof.

The completed verification has the following scope:

Check	Count
Strict/non-strict bound-comparison grid entries	5760
One-variable elimination output entries	1184
Complete finite sums, including signed weights	71
First-compression cell masses	32
Second compression after a new aligned mask	2
End-to-end HV/magnitude instances, exact inclusion–exclusion	54
Forced deep recursion instances	2
Magnitude–HV affine equivalence instances	6
Geometric face-weight contraction checks	358
Independent FastHVChan cross-checks	4

The predicate tests exercise increasing and decreasing cutoffs, both inequality senses, ties, reversed argument order, and an alternating same-variable filter. Static numerical elimination is checked up to eight variables. Compression is tested at the full eight variables needed in 4D. The geometric contraction checks use floating-point evaluation only for the fixed algebraic face weights, with a tolerance; the measure and elimination comparisons use exact arithmetic.

A reproducible twelve-point instance shows the effect of scheduling:

Schedule	Measure	Generations	Compressions	Peak terms
Default block rule	HV	1	1	8
Default block rule	Magnitude	1	1	24
One-level stress rule	HV	4	11	451
One-level stress rule	Magnitude	4	11	473

“Generations” is the maximum along a path; “compressions” counts all branches. Both schedules give 1492 for HV and 1215/4 for magnitude on this instance. The one-level override is a closure stress test and is *not* the schedule covered by the asymptotic theorem. These small tests establish implementation checks, not an empirical measurement of the 4/3 exponent or a speed advantage over existing code.

13 Numba implementation and measured comparison

The directory `NumericalChanHV4D` also supplies `numerical_chan4_numba.py`, a four-dimensional entry point to the shared `float64` engine in `NumericalChanHVND`. The shared engine compiles numerical array scans, prefix sums, cutoff searches, normalization, and suffix masks with Numba. Geometric recursion and structural term construction remain Python; it is a hybrid backend. The exact standard-library implementation remains independently usable. Neither `fastmath` nor parallel execution is enabled.

The reproducible benchmark compares the numerical Python and Numba backends with the repository’s unmodified Python implementations of Chan Section 4.2 ($d/3$) and Section 2 ($d/2$). The baseline commit is `08539aa99e94`; neither baseline uses Numba. All timed inputs are floating point. Dominance prefiltering is disabled, and input preprocessing and coordinate transforms are included. For magnitude, the references use the affine HV transformation.

Each entry below is the median of three calls after one untimed first call. Workers run sequentially. A 35-second limit covers the entire worker, including startup, its first call and all repeats; “timeout” is not a per-call runtime. Raw repeats and first-call times are in the accompanying JSON/CSV. No ratio is assigned to a timeout.

Case	Numerical Python	Numba	Chan 4.2	Chan 2
sphere, 12, HV	0.028069	0.041134	0.012223	0.0017601
sphere, 12, Mag	0.028301	0.043302	0.0126	0.0019805
sphere, 24, HV	1.4637	2.3398	0.027249	0.0054424
sphere, 48, HV	timeout	timeout	0.10325	0.018492
grid, 12, HV	0.0046516	0.0064536	0.0030063	0.0005476
grid, 12, Mag	0.004033	0.006346	0.0034868	0.000561
grid, 24, HV	0.045366	0.076442	0.0065916	0.001393
grid, 48, HV	0.074685	0.1358	0.0018386	0.0009735

All times are seconds. The ND overview uses spherical, mutually nondominated fronts; the full data additionally includes tied-grid inputs and magnitude evaluations. These small cases do not measure asymptotic exponents.

For completed matched ordinary-HV cases in 4D, the median ratio Numba numerical / Python numerical was 1.60x (5 cases). The median ratio Numba numerical / the repository’s Python Section 4.2 implementation was 11.60x (5 cases). Ratios above one mean the Numba version was slower. These are medians of per-case ratios for this small benchmark set. They do not support a practical speedup claim for this four-dimensional implementation.

Across completed comparisons the maximum scaled disagreement was 3.31×10^{-16} , where scaling divides by $\max(1, |\text{reference value}|)$. This is an observed check, not a uniform error bound.

Fresh-cache first-use probes (module imports excluded) gave:

Dimension	First use (s)	Second use (s)
2	2.5352	9.72e-05
3	2.496	0.0001481
4	1.9391	0.0030841

The first call includes both compilation and computation; it is not pure compiler time.

Environment: CPython 3.12.14, NumPy 2.5.3, Numba 0.68.0, Windows.

14 What extends to dimensions six and eight

The numerical term class and its four slots per pair are dimension independent. In dimension d , compression introduces $2d$ variables and eliminates the old d , in pairs when d is even. The raw constant is

$$C_d = \prod_{p=d+1}^{2d} B_p.$$

It can be very large, but is independent of the number of input points. For $d = 6, 8$, the compression dimensions are respectively 12, 16.

The geometric potential uses active triples exactly as before, now with weights $2^{\sum \text{pos}/d}$. A cyclic level decreases potential by $2^{-3/d}$; a full d -axis round creates at most 2^d descendants and decreases potential by a factor eight. Taking approximately $\frac{1}{4} \log_2 N$ levels per locally chosen block gives

$$T_d(N, S) \leq O(N^{1/4})T_d(O(N^{1-3/(4d)}), C_d S) + \tilde{O}_d(SN^{5/4}).$$

Since $5/4 < d/3$ for $d \geq 4$, the same analysis gives $\tilde{O}_d(n^{d/3})$: $\tilde{O}(n^2)$ in six dimensions and $\tilde{O}(n^{8/3})$ in eight dimensions. This describes the mathematical extension. The standalone reference solver in this directory is four-dimensional. The companion directory `NumericalChanHVND` now implements and tests every dimension from two through ten, including 12-, 16-, and 20-variable compression. Its own report distinguishes small structured validation from practical performance on large coupled inputs.

15 The resulting algorithmic target

The requested $\tilde{O}(n^{4/3})$ numerical formulation is obtained without the unproved global batching step from the earlier faster-algorithm investigation. Its complexity follows from

Chan’s geometric cuts plus a proved finite-array compression and a schedule that explicitly pays for term growth.

The reusable core is small: four monotone slots per pair, one prefix-sum eliminator, and block-mass pushforward. The geometric recursion is binary, organized into the coordinate pairs $(1, 2)$ and $(3, 4)$. Further practical work can specialize these fixed operations or compile their arrays, while keeping the same correctness and complexity argument.

References

- [1] T. M. Chan. *Klee’s Measure Problem Made Easy*. Proceedings of FOCS 2013, pp. 410–419. DOI: 10.1109/FOCS.2013.51. Author’s August 2013 version, Sections 4.1–4.2: https://tmc.web.engr.illinois.edu/easyklee8_13.pdf.
- [2] M. T. M. Emmerich. *The Magnitude of Dominated Sets: A Pareto Compliant Indicator Grounded in Metric Geometry*. arXiv:2604.18147, 2026. <https://arxiv.org/abs/2604.18147>.
- [3] M. Emmerich and collaborators. *FastHVChan*: source code and implementation report. Repository snapshot 08539aa99e94, accessed 4 October 2026. <https://github.com/emmerichmtm/FastHVChan>.
- [4] M. Emmerich and collaborators. *HV4DMagnitude*: numerical prefix formulation, audits, and implementation. Repository snapshot 0c668e765202, accessed 4 October 2026. <https://github.com/emmerichmtm/HV4DMagnitude>.
- [5] M. T. M. Emmerich. *Pair-Elimination and Product-Measure Simplification of Chan’s Fixed-Dimensional Hypervolume Algorithm*. Working report, `paper/magnitude_chan_pair_elimination_report.tex`, in the HV4DMagnitude snapshot cited above. Also inspected: `paper/magnitude_chan4_report_closed.tex`.
- [6] *HV468DMagnitude*: pair-elimination implementation and audit material, including `PAIR_ELIMINATION_COMPLEXITY.md` and `AUDIT_RESPONSE.md`, in the HV4DMagnitude snapshot above. <https://github.com/emmerichmtm/HV4DMagnitude/tree/main/HV468DMagnitude>.